

INSERT YOUR COVER PAGE HERE

Table of Contents

1. Problem Identification	3
1.1 Mapping to Project Objectives	4
2. Optimization Problem Formulation	4
2.1 The volume-rendering operator $R(\theta; \pi)$	5
2.2 Why first-order stochastic methods	5
2.3 The role of positional encoding	5
3. Optimization Methods to be Used	6
3.1 Scene Representations	6
3.2 Optimizers (applied to NeRF)	6
3.3 Loss Formulations	8
4. Analysis Plan	8
5. Planned Improvements (Stretch Goals)	9
6. Dataset, Compute, and Software	10
7. Work Distribution and Deliverable	10

1. Problem Identification

3D scene reconstruction from a small set of 2D photographs is a foundational task in computer vision and AI. Given a collection of images captured from different viewpoints, the goal is to recover a representation of the underlying scene that allows new, previously unseen views to be rendered with high fidelity. The task underpins applications across robotics, augmented and virtual reality, autonomous driving, virtual production, and digital heritage preservation.

Two recent approaches have defined the state of the art:

- **Neural Radiance Fields (NeRF)**, introduced in 2020, represents the scene as a continuous function parameterised by a small multi-layer perceptron that maps a 3D point and a viewing direction to a colour and a density.
- **3D Gaussian Splatting**, introduced in 2023, represents the scene as a large collection of explicit 3D Gaussian primitives, each with a position, scale, rotation, opacity, and colour.

Both approaches frame scene reconstruction as a continuous, non-convex optimization problem: find the parameters of the chosen representation such that, when rendered from the known camera viewpoints, they best reproduce the captured photographs.

This project formulates and studies that optimization problem in depth. We are interested not only in producing visually convincing 3D reconstructions, but in understanding how the choice of optimizer, loss function, regularisation, and representation affects convergence behaviour, training stability, and final reconstruction quality.

From an optimization standpoint, the problem is interesting because it combines several characteristics that classical methods handle poorly. It is **non-convex**, because the rendering operator composes the differentiable volume-rendering equation with the non-linear MLP (NeRF) or with the non-linear Gaussian rasterisation pipeline (Gaussian Splatting); even with a convex per-pixel loss, the composed objective in θ is non-convex with many local minima and saddle points. It is **high-dimensional**, with $|\theta|$ on the order of 10^5 parameters for a compact NeRF MLP and 10^6 to 10^7 for Gaussian Splatting on a typical scene. It is **stochastic**, because at each iteration only a small subset of rays (around 10^3 to 10^4 out of millions per scene) can be sampled; the per-step gradient is therefore a noisy estimate of the true gradient. And it is **ill-conditioned in pixel space**: textured regions of the scene produce large gradients while textureless regions produce nearly none, so any optimizer that does not adapt or sample carefully will either over-fit easy regions or stagnate on hard ones.

The choice of optimizer is therefore not cosmetic. Methods that converge cleanly on convex problems can fail or stagnate here, and methods designed for non-convex stochastic problems exhibit substantially different convergence dynamics in practice. Documenting and explaining these differences on a concrete, visualisable problem is the central contribution of this project. The problem is also still an active area of research: the original NeRF paper triggered a still-growing body of work on accelerated variants (Instant-

NGP, Plenoxels), alternative representations (3D Gaussian Splatting, triplane methods), extensions to dynamic scenes (4D Gaussian Splatting, 2024-2025, which augments each Gaussian primitive with a temporal evolution model), and improved optimization recipes (curriculum sampling, adaptive density control, learning-rate restarts). The design space of methodological choices to make and defend is non-trivial, and this project deliberately scopes itself to the static-scene case so that the optimization-side comparison can be made under controlled conditions.

1.1 Mapping to Project Objectives

The five project objectives (formulate; implement; analyse; compare; propose improvements) are addressed by Sections 2, 3, 4, 4, and 5 respectively. The remainder of this document develops each in detail.

2. Optimization Problem Formulation

Let θ denote the parameters of the chosen scene representation: - For NeRF, θ is the set of weights and biases of a small MLP. - For 3D Gaussian Splatting, θ is the collection of 3D Gaussian positions, scales, rotations, opacities, and colours.

Let $R(\theta; \pi)$ denote the differentiable rendering operator: given the parameters θ and a camera pose π , it produces a synthesised image. Let $\{(I_i, \pi_i)\}_{i=1..N}$ be the captured photographs and their associated camera poses (recovered by Structure-from-Motion).

The reconstruction problem is then:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(R(\theta; \pi_i), I_i) + \lambda \Omega(\theta)$$

where L is a per-image loss that measures the discrepancy between the rendered and observed images, and $\Omega(\theta)$ is a regularisation term on the parameters with weight $\lambda \geq 0$ (typically the squared L2 norm of the network weights for the NeRF MLP, or a density / opacity regulariser for Gaussian Splatting; specific candidates are listed below). Because the rendering operator is differentiable with respect to θ , the problem admits gradient-based optimization, although the loss landscape is non-convex and high-dimensional.

The formulation choices we will make and defend explicitly:

- **The image loss L .** Candidates include L1, L2, SSIM, and convex combinations of these. Each has known effects on which image details are preserved and how robust the optimization is to outliers.
- **The regularisation term $\Omega(\theta)$.** Candidates include weight decay (for the NeRF MLP), and density / opacity regularisers and adaptive density control heuristics (for Gaussian Splatting).
- **The view- and ray-sampling strategy.** Which rays or which views are presented to the optimizer at each iteration. Uniform random, importance sampling by current reconstruction error, and curriculum schedules are options.

- **The parameterisation choices.** Positional encoding frequency for NeRF; initial Gaussian distribution and density-control thresholds for Gaussian Splatting.

These choices are part of what the project will study, not fixed in advance.

2.1 The volume-rendering operator $R(\theta; \pi)$

For each ray cast through a pixel, the rendering operator samples N positions along the ray between near and far planes, queries the scene representation at each position to obtain a density $\sigma_k \geq 0$ and an RGB colour $c_k \in [0, 1]^3$, and composites them into a per-pixel colour using the discretised volume-rendering equation:

$$C(\mathbf{r}) = \sum_{k=1}^N T_k (1 - \exp(-\sigma_k \delta_k)) c_k, \quad T_k = \exp\left(-\sum_{j < k} \sigma_j \delta_j\right)$$

where $\delta_k = t_{\{k+1\}} - t_k$ is the distance between consecutive samples and T_k is the accumulated transmittance up to sample k . The operator is differentiable with respect to every σ_k and c_k , and through the representation also with respect to θ . The full objective is therefore amenable to gradient-based optimization via reverse-mode automatic differentiation (PyTorch autograd composes the gradient through the loss, the renderer, the encoding, and the MLP in one backward pass).

2.2 Why first-order stochastic methods

Given the properties of the problem (non-convex, high-dimensional, stochastic, ill-conditioned), first-order stochastic methods are the appropriate family of solvers and the focus of the implementation work.

Second-order methods (Newton, BFGS, Gauss-Newton) are infeasible at this scale because the Hessian $\nabla^2_{\theta} L$ cannot be stored explicitly: for $|\theta| = 10^5$, a dense Hessian already occupies tens of gigabytes. Quasi-Newton methods such as L-BFGS are sometimes feasible but their assumption of relatively smooth, deterministic objectives is violated by the high gradient noise of mini-batch sampling. First-order methods, by contrast, store only first-moment state (SGD, momentum) or both first- and second-moment state (Adam, AdamW), scale linearly in $|\theta|$, and tolerate stochasticity through momentum smoothing or per-parameter adaptation.

2.3 The role of positional encoding

A naively parameterised MLP that takes a raw 3D point as input cannot fit high-frequency scene detail: the implicit bias of small fully-connected networks is toward low-frequency functions of their inputs. To address this, we wrap the input coordinate in a fixed (non-learned) positional encoding $\gamma(x)$:

$$\gamma(x) = [x, \sin(2^0 \pi x), \cos(2^0 \pi x), \dots, \sin(2^{L-1} \pi x), \cos(2^{L-1} \pi x)]$$

with L between 8 and 14 frequency bands. The encoding adds no parameters to be optimised; it changes the input to the MLP so that high-frequency variations in the scene become easier for the optimizer to discover. The choice of L materially affects both

reconstruction quality and the conditioning of the optimization problem: too few bands and the model cannot represent fine detail; too many and the model overfits noise and training becomes unstable. This is one of the formulation choices we will study empirically.

3. Optimization Methods to be Used

We will implement and compare methods along two axes: the choice of scene representation, and the choice of optimizer.

3.1 Scene Representations

NeRF, implemented from scratch. We will write a compact implementation of the NeRF representation and the volume rendering integration in PyTorch. The implementation will be small (on the order of a few hundred to a few thousand lines of clear Python), so that every optimization choice remains under our control. This is the heart of the optimization study.

3D Gaussian Splatting, using a published reference implementation. We will use an open-source Gaussian Splatting implementation as a contemporary point of comparison. Reimplementing the custom CUDA rasteriser is out of scope; the contribution of our project is to study how the optimization behaviour of NeRF compares with that of Gaussian Splatting on the same scenes, and how our optimization choices on the NeRF side affect the gap.

3.2 Optimizers (applied to NeRF)

We will implement the following five optimizers ourselves on top of PyTorch’s automatic differentiation, plus a learning-rate schedule, and compare them on the NeRF reconstruction problem. For each method we give the per-iteration update rule, what aspect of the optimization problem the method is designed to address, and what its inclusion contributes to the comparison.

Stochastic Gradient Descent (SGD).

$$\theta_{t+1} = \theta_t - \eta g_t, \quad g_t = \nabla_{\theta} \mathcal{L}(\theta_t; \text{batch}_t)$$

The simplest stochastic first-order method and the reference baseline. Convergence is slow on ill-conditioned problems because the step direction is the raw stochastic gradient, with no smoothing or per-parameter scaling. Its inclusion sets the floor of what every other method must improve upon.

SGD with classical momentum.

$$\begin{aligned} v_{t+1} &= \beta v_t + g_t \\ \theta_{t+1} &= \theta_t - \eta v_{t+1} \end{aligned}$$

Maintains a running exponential average v of past gradients (momentum coefficient β typically 0.9). Smoothing across iterations damps gradient noise and accelerates convergence along consistent descent directions (the canonical “ravines” of the loss

surface). Demonstrates the effect of first-moment estimation on a stochastic non-convex problem.

SGD with Nesterov accelerated gradient.

$$\begin{aligned} v_{t+1} &= \beta v_t + \nabla_{\theta} \mathcal{L}(\theta_t - \eta \beta v_t; \text{batch}_t) \\ \theta_{t+1} &= \theta_t - \eta v_{t+1} \end{aligned}$$

Evaluates the gradient at a *look-ahead* point $\theta_t - \eta \beta v_t$ rather than at the current θ_t . In convex settings this yields a provably better convergence rate ($O(1/k^2)$ vs $O(1/k)$); in stochastic non-convex settings the theoretical advantage is weaker but the look-ahead still tends to reduce overshoot near sharp valleys. Demonstrates the practical effect of acceleration on a problem where classical guarantees do not apply.

Adam (Kingma & Ba, 2014).

$$\begin{aligned} m_{t+1} &= \beta_1 m_t + (1 - \beta_1) g_t \\ v_{t+1} &= \beta_2 v_t + (1 - \beta_2) g_t^2 \\ \hat{m}_{t+1} &= m_{t+1} / (1 - \beta_1^{t+1}) \quad (\text{bias correction}) \\ \hat{v}_{t+1} &= v_{t+1} / (1 - \beta_2^{t+1}) \\ \theta_{t+1} &= \theta_t - \eta \hat{m}_{t+1} / (\sqrt{\hat{v}_{t+1}} + \varepsilon) \end{aligned}$$

Combines a momentum-like first moment m with a per-parameter second-moment estimate v (typically $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$). The per-parameter adaptive scaling $1/\sqrt{\hat{v}}$ directly addresses the ill-conditioning of the problem: parameters whose gradients are consistently large get smaller effective steps, and vice versa. Demonstrates the impact of second-moment adaptation on a stochastic, ill-conditioned problem.

AdamW (Loshchilov & Hutter, 2017).

$$m_{t+1}, v_{t+1}, \hat{m}_{t+1}, \hat{v}_{t+1} \text{ as in Adam,} \quad \theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \varepsilon} + \lambda \theta_t \right)$$

Modifies Adam by decoupling the weight-decay term $\lambda \cdot \theta_t$ from the adaptive gradient step. In standard Adam, adding L2 regularisation to the loss interacts with the second-moment estimate v in ways that effectively cancel the regularisation for large-gradient parameters. AdamW separates the two, which makes weight decay function as intended. Demonstrates the interaction between regularisation and adaptive optimization.

Learning-rate schedule. On top of the best-performing optimizer, we will apply a cosine-annealing schedule with a short warmup:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t}{t_{\text{warmup}}}, & t \leq t_{\text{warmup}} \\ \eta_{\max} \cdot \frac{1}{2} \left(1 + \cos \left(\pi \frac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}} \right) \right), & t > t_{\text{warmup}} \end{cases}$$

Schedules of this form have become standard in deep learning because they explore the loss surface aggressively early in training (large η) and refine near a minimum late in

training (small η). Their effect on this non-convex stochastic problem is one of the things the comparison will quantify.

Why implement from scratch. Implementing the optimizers ourselves is what makes this a study of computational optimization rather than a benchmark of pre-existing library code. Every gradient step, momentum buffer update, and bias-correction term is code we wrote, can inspect, can modify, and can defend in the analysis. It is also what makes the comparison fair: identical numerical conventions, identical edge-case handling, and identical state initialization across all five methods, none of which can be guaranteed when comparing different library implementations against each other.

What the comparison will produce. Under identical iteration budgets, data sampling, and initialization, the comparison produces per-method rankings by convergence speed, final quality at a fixed budget, stability across seeds, and learning-rate sensitivity. These quantities are the basis of the methodological justification required by project objectives.

3.3 Loss Formulations

We will compare:

- L1 loss between rendered and observed pixels
- L2 loss
- SSIM loss
- A weighted combination of L1 and SSIM (as used in the Gaussian Splatting paper)

4. Analysis Plan

For each combination of representation, optimizer, and loss, we will measure and report:

Convergence behaviour:

- Training loss as a function of iteration count
- Rendered-image PSNR and SSIM on held-out views as a function of iteration count
- Wall-clock time to reach a fixed target quality

Stability:

- Sensitivity to random initialization across multiple seeds
- Sensitivity to learning rate (sweeps)
- Behavior on scenes of different difficulty (textured vs textureless, dense vs sparse view coverage)

Final quality:

- PSNR and SSIM on held-out test views
- Qualitative side-by-side comparison of rendered novel views

All numerical comparisons will be averaged across at least three seeds, and we will report variability alongside means.

5. Planned Improvements (Stretch Goals)

Based on the comparison results from Section 4, we will investigate the four improvements below. Each is motivated by a specific property of the optimization problem identified in Section 2.

We frame these as **stretch goals** rather than firm commitments. The baseline study (Sections 3 and 4) is the core deliverable; the improvements are evaluated as time and the observed bottlenecks allow. At minimum we will deliver **one** improvement implemented and reported as a full ablation in Phase 2; the selection will be guided by which characteristic of the optimization problem proves most limiting in the baseline runs (for example, if convergence stagnates on textureless regions we prioritise adaptive view sampling; if the loss curves show oscillation in the late phase we prioritise learning-rate restarts). Any remaining improvements will appear in the writeup as discussed-but-not-implemented future work.

Adaptive view sampling. Standard NeRF training samples rays uniformly across the training images. This wastes computation on already-well-reconstructed regions and underservices regions with high residual error (the ill-conditioning property of Section 2.2). We will replace uniform sampling with importance sampling based on the current per-pixel reconstruction error:

$$p_i \propto (e_i + \varepsilon)^\alpha$$

where e_i is a running estimate of the per-pixel squared error, $\varepsilon > 0$ ensures all pixels remain reachable, and $\alpha \in [0, 1]$ controls the steepness of the bias. This modifies the stochastic gradient estimator (different importance weights, different effective batch composition) and is expected to reduce the number of iterations needed to reach a given target PSNR.

Multi-scale (coarse-to-fine) training. A direct way to mitigate the non-convexity of the loss is to begin optimization on a smoother, lower-resolution version of the problem and progressively refine. We will train initially at reduced image resolution and double it at fixed iteration milestones:

$$H_t = \min(H_{\min} \cdot 2^{\lfloor t/T_{\text{stage}} \rfloor}, H_{\max})$$

The low-resolution objective has a smoother landscape (fewer high-frequency details to fit) and the converged low-resolution solution acts as a warm start for the next stage. This is a curriculum-optimization approach with measurable effect on overall time-to-convergence.

Learning-rate restarts (warm restarts, SGDR). To escape sharp local minima in the non-convex landscape, we will replace the single cosine-annealing schedule of Section 3.2 with a cyclic cosine schedule (Loshchilov & Hutter, “SGDR”):

$$\eta_t = \eta_{\min} + 1/2 (\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\pi \frac{t \bmod T_i}{T_i} \right) \right)$$

where T_i is the length of the i -th cycle (optionally doubled each cycle). At the end of each cycle the learning rate jumps back to $\eta_{\max}$, allowing the optimizer to leave its current

basin and potentially settle in a better one. The effect on final quality and on the variance across seeds will be measured.

Perceptual loss. Pixel-wise losses (L1, L2) are poorly aligned with human perception of image quality: an image that is uniformly slightly off in colour can have the same MSE as one with a sharp edge displaced by a pixel, but the two look very different. We will augment the pixel-wise loss with a feature-space loss based on the activations of a pretrained image classifier ϕ :

$$\mathcal{L}_{\text{perc}}(I_{\text{pred}}, I_{\text{gt}}) = \sum_{l \in \text{layers}} w_l \|\phi_l(I_{\text{pred}}) - \phi_l(I_{\text{gt}})\|_2^2$$

This does not change the dimensionality of θ , but it changes the gradient and therefore the optimization dynamics. The expected effect is improved perceptual quality (lower LPIPS) at possibly similar or slightly worse PSNR; the trade-off itself is informative.

Each improvement will be evaluated against the best-performing baseline configuration (the best optimizer + loss combination identified in the main comparison) as an isolated ablation, with effect measured quantitatively in PSNR, SSIM, LPIPS, and time-to-target-quality. Improvements will not be combined into a single “super-recipe”; the goal is to attribute effect to cause, not to maximise a single number.

6. Dataset, Compute, and Software

We will use one or two self-captured real-world scenes (smartphone, around 30 to 60 images each, processed via COLMAP for Structure-from-Motion) plus at least one standard NeRF-synthetic scene for comparison with published numbers.

The stack is Python and PyTorch, with custom optimizer and volume-rendering code for NeRF, an open-source reference implementation for Gaussian Splatting, and COLMAP for camera calibration.

7. Work Distribution and Deliverable

The workload is split equally across the project's several areas, with joint participation in code review, tutorials, and synchronization meetings. The final deliverable is a Jupyter Notebook accompanied by a written report derived from it.